

Snake Game Project

This report presents the development of a Python Snake game.

Stages of the program development included;

- Problem recognition.
- Problem definition.
- Program design.
- Program coding.
- Program testing and debugging
- Program implementation and Maintenance
- Program documentation

1. Problem Recognition

We wanted to build a simple game that we could make completely on our own using Python. Before the coding started the requirement for what we needed to build included ;

- A snake that moves continuously around a grid-based window.
- The snake grows by one segment each time it eats a food item.
- The game ends if the snake hits a wall or collides with its own body.
- The player can control direction using the arrow keys while not able to instantly reverse into itself.
- A score is tracked and displayed and the player can restart after losing the game.

2. Problem Definition

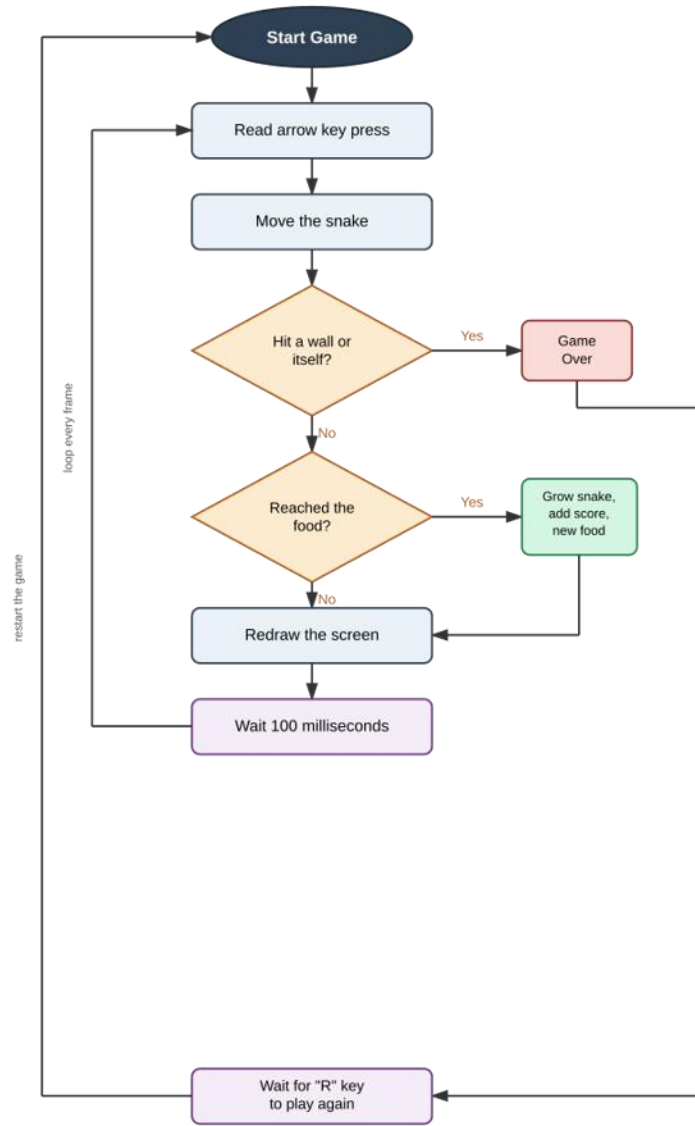
Once we understood the problem, we wrote down exactly what the program needed to do:

- Show a snake on a grid (I used a 25x25 grid of 25-pixel tiles).
- Let the player move it with the arrow keys, but not let it turn back on itself.
- Move the snake automatically 10 frames per second so that it's always moving.
- Put food somewhere random on the grid, and when the snake eats it, make the snake longer and add to the score.
- End the game if the snake hits a wall or hits its own body, and play a sound when that happens.
- Show the score while playing, and show "Game Over" with the score at the end.
- Let the player press R to play again after losing.

2.1 Input, Process, Output

To keep things clear, we broke the main parts of the game down into what goes in, what the program does with it, and what comes out:

Input	Process	Output
Arrow key press	The program checks the key and changes the direction but won't let the snake reverse into itself	The snake's direction changes
Snake's current position and direction	Each body part moves to where the part in front of it was then the head moves one step	The snake moves forward on screen
Head position vs walls/body	The program checks if the head has hit a wall or its own body	Game over message shown and game-over sound plays
Head position vs food position	The program checks if the head is on the same tile as the food	Snake gets longer, score goes up, new food appears, pickup sound plays
"R" key after losing	The program resets all the variables back to the starting values	A brand new game starts



3. Program Design

3.1 Data that was needed

- A Tile class that just stores an x and y position. used this for the snake's head, every part of its body, and the food, so they could all be treated the same way.

- A variable for the snake's head, and a list for the body parts.
- velocityX and velocityY to keep track of which direction the snake is moving.
- A score variable and a game_over variable (True or False).

3.2 What it looks like

The game window is 625x625 pixels (25 tiles x 25 pixels), and it opens in the middle of the screen.

The snake and its body are green rectangles, the food is a red rectangle, the score shows in the top left in purple and when the game ends it shows a message in the middle of the screen.

3.3 How we split up the code

Instead of writing everything in one big block, we split the game into separate functions that each do one job:

- Setting up the window and loading the sounds.
- reset_game() - resets everything back to the start.
- change_direction() - handles key presses.
- move() - handles all the game logic (moving, checking collisions, checking food).
- draw() - redraws everything on screen and keeps the game loop going.

4. Program Coding

After planning it out, it was coded in roughly this order: the Tile class first, then the setup code and sounds, then reset_game(), then change_direction(), and finally move() and draw(), since those two are the heart of the whole game. Here's roughly what happens every frame:

```
draw():
    move()
        if the game is already over, do nothing
        if the snake went off the screen -> game over
        if the snake hit itself -> game over
        if the snake reached the food:
            grow the snake, move the food, add 1 to score
        move the body (each part follows the one in front)
        move the head in the current direction
    clear the screen and redraw everything
    show the score, or 'game over' if it's over
    do this again in 100ms
```

One thing we had to be careful about: in change_direction(), we only let the direction change if it's not the exact opposite of where the snake is already going. That's what stops the player from turning the snake straight back into its own neck.

5. Program Testing and Debugging

We tested the game bit by bit as it was added each feature. instead of writing everything first and testing at the end. That made it way easier to know exactly which part broke when something went wrong.

5.1 Test table

	What was tested	What we expected	What happened
1	Pressing an arrow key	Snake should turn that way	Worked fine
2	Trying to reverse the snake into itself	Should just be ignored	Didn't work at first, fixed it by checking velocity before turning
3	Snake eating food	Snake grows, score +1, new food shows up	Worked fine
4	Snake hitting its own body	Game should end	Worked fine
5	Snake hitting the wall	Game should end	Worked fine
6	Pressing R after game over	New game should start	Worked fine

6. Program Implementation and Maintenance

To actually run the game, you just need Python with tkinter and pygame installed, plus the three sound files (pickup.mp3, game_over.mp3, game_start.mp3) in the same folder as the script. Then you just run the file and it opens the game window.

Things we would still want to fix or improve if we kept working on this:

- The game speed never changes, so it might be more fun if it sped up as the score gets higher.
- There's no pause button or difficulty settings yet.

7. Program Documentation

This is a technical or programmer oriented documentation explaining the technical aspects on how the program works and how it was developed. For this project that includes:

- Comments and clear variable names in the code itself, like `snake_body` and `change_direction` so it's easier to follow.